

Can policy gradient replicate the exact soccer solver?

REINFORCE, A2C and PPO, trained by self-play and by fictitious play, scored against the exact solution of the discounted 100-step soccer game. Every number below is read from the experiment files in the repository.

The short answer: not yet.

- All six learners (REINFORCE, A2C and PPO, each trained by self-play and by fictitious play) stay far from the equilibrium: exploitability 0.76 to 0.95, where the exact solution is 0.
- Fictitious-play training wins more against a random player but loses to the exact equilibrium; self-play training ties it more often.
- No learner is mirror-symmetric (mirror gap 0.44 to 0.81, against 0 for the exact solution).

These are 3 seeds at one untuned budget, so this shows that these settings did not converge, not that policy gradient cannot.

1. What the professor said, and what was done

The group's questions and his answers (relayed in the group channel) set the design of everything below.

question	his answer	what was done
Which environment for policy gradient?	Any one where the solver is right and the comparison is meaningful; fewer mixed-equilibrium stages is better this time.	The deterministic A10 board, which has no mixed stage games, is the main environment; the random move-order board is kept as an earlier comparison.
Discounted or finite-horizon objective?	Policy gradient can only approximate finite-horizon discounted rewards, so both.	Discount 0.9 over 100 steps, a tie at the end, and the remaining step count as a network input.
How should fictitious play be used?	No: there is no explicit game solving in policy gradient; fictitious play is how policy gradient solves things.	Each player is trained by policy gradient against the average of the opponent's past policies. Matrix-level fictitious play is kept only as a side check (section 7).
How to define the win rate?	Repeated game plays, and just count the wins.	1,000 repeated games from the kickoff against a random player, the exact Nash policy and the exact best response; wins, ties and losses are all counted.

Continuous best-response methods now?	No, those are for DQN, not policy gradient, which can be adapted to continuous actions directly.	Not applied to policy gradient. The dog game uses an angle-radius policy trained directly (section 8, provisional); the bisection, finite-difference and quadratic methods exist separately for DQN.
---------------------------------------	--	--

2. Setup

The environment is the deterministic A10 soccer game on a 7×5 board with a three-cell goal; it has no mixed-strategy stage games, which the professor preferred for this comparison. The objective is **discounted** ($\gamma = 0.9$) over a **100-step horizon**, a tie if nobody scores, and the network receives the **remaining step count** as an input. The reference is the exact discounted backward induction over the same 100 steps (`soccer_nash/finite_horizon.py`), whose equilibrium has exploitability 0 and ties itself every game.

3. The learners

algorithm	update
REINFORCE	plain discounted returns, no baseline
A2C	10-step bootstrapped advantage from a learned critic
PP0	GAE ($\lambda = 0.95$), clipped ratio 0.2, 4 epochs

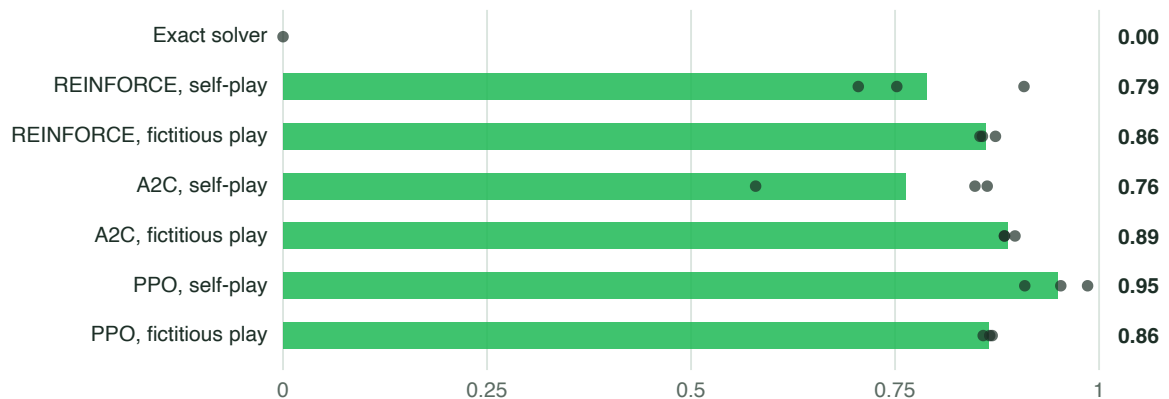
Self-play trains both current networks against each other. **Fictitious play** is how policy gradient solves the game here, with no matrix solving: each player keeps improving its own network by policy gradient against the *empirical average* of the opponent's past policies (frozen snapshots, one drawn per episode), and reports the average of its own snapshots. All learners use learning rate 10^{-3} , 64 episodes per batch, 2,000 iterations, an entropy bonus of 0.01, and a 64×64 network; none was tuned.

4. How they are scored

- **Exploitability**: the discounted value both exact best responses earn from the kickoff, summed. Zero exactly at an equilibrium.
- **Wins, ties and losses**: counts over 1,000 repeated games from the kickoff, against a random player, the exact Nash policy and the exact best response.
- **Mirror gap**: how far the two players' policies are from being mirror images (flip the board, swap the players), averaged over all states at five points in the game.

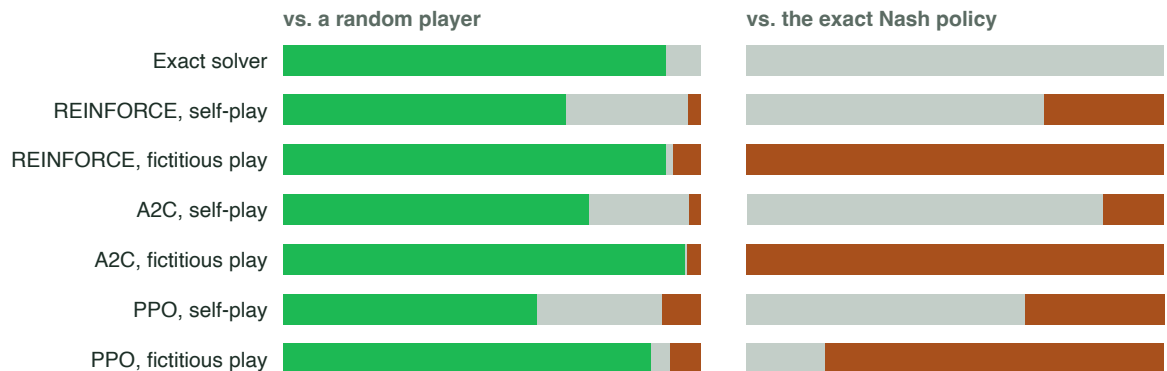
5. Results

learner	exploitability (mean $\pm$ sd)	W/T/L vs. random	vs. exact Nash	vs. best response	mirror gap
Exact solver	0.000 $\pm$ 0.000	0.92/0.08/0.00	0.00/1.00/0.00	0.00/1.00/0.00	0.00
REINFORCE, self-play	0.788 $\pm$ 0.106	0.68/0.29/0.03	0.00/0.71/0.29	0.00/0.00/1.00	0.66
REINFORCE, fictitious play	0.861 $\pm$ 0.010	0.92/0.02/0.07	0.00/0.00/1.00	0.02/0.00/0.98	0.44
A2C, self- play	0.763 $\pm$ 0.160	0.73/0.24/0.03	0.00/0.85/0.15	0.02/0.00/0.98	0.53
A2C, fictitious play	0.888 $\pm$ 0.008	0.96/0.00/0.03	0.00/0.00/1.00	0.01/0.00/0.99	0.44
PP0, self- play	0.949 $\pm$ 0.039	0.61/0.30/0.09	0.00/0.67/0.33	0.00/0.00/1.00	0.81
PP0, fictitious play	0.864 $\pm$ 0.006	0.88/0.05/0.07	0.00/0.19/0.81	0.00/0.00/1.00	0.48



Exploitability by learner: bar = mean, dots = individual seeds.

win tie loss



Two patterns repeat. Fictitious-play training beats a random player more often (0.88 to 0.96) but loses almost every game to the exact equilibrium. Self-play training wins less often against random and ties

the equilibrium more often: it plays more cautiously. Neither is close to equilibrium play, and every learner loses almost every game to the exact best response.

Does more training help?

Self-play with **8,000** iterations instead of 2,000 (2 seeds per learner; every other setting unchanged). The fictitious-play runs were too slow to repeat at this length.

learner	exploitability, 2,000 it. (3-seed mean)	exploitability, 8,000 it. (each seed)	W/T/L vs. random (8,000 it.)	W/T/L vs. exact Nash (8,000 it.)
REINFORCE, self-play	0.788	0.49 / 0.91	0.91/0.08/0.02	0.00/0.38/0.62
A2C, self- play	0.763	0.44 / 0.50	0.79/0.20/0.01	0.00/0.79/0.21
PPO, self- play	0.949	0.91 / 0.96	0.97/0.01/0.02	0.00/0.00/1.00

REINFORCE ends clearly lower on 1 of 2 seeds; A2C ends clearly lower on 2 of 2 seeds; PPO ends clearly lower on 0 of 2 seeds; the best single run reaches 0.44, still far from 0. With two seeds per learner this is a trend, not a result.

6. Earlier experiments

Before the professor's answers, the same algorithms were run on the stationary discounted game (no step count), from the kickoff or from random starting states.

setup	algorithm	exploitability	mean equilibrium regret	action agreement
A10 board, stationary, kickoff starts	REINFORCE	0.83	0.124	0.28
A10 board, stationary, kickoff starts	A2C	0.83	0.122	0.29
A10 board, stationary, kickoff starts	PP0	0.87	0.167	0.25
random board, stationary, kickoff starts	REINFORCE	0.73	0.056	0.44
random board, stationary, kickoff starts	A2C	0.78	0.058	0.45
random board, stationary, kickoff starts	PP0	0.85	0.108	0.32
random board, stationary, exploring starts	REINFORCE	0.80	0.027	0.71
random board, stationary, exploring starts	A2C	0.82	0.030	0.70
random board, stationary, exploring starts	PP0	0.91	0.072	0.51

Exploring starts raise per-state accuracy (action agreement) but not play from the kickoff.

7. Fictitious play on the exact stage games

As a side check, fictitious play was also run on the exact 4×4 stage matrices of the random move-order board (2,380 states, 94 with no pure saddle). On rock-paper-scissors best-response dynamics cycle forever while fictitious play settles at 1/3 each; on the soccer stage games it recovers the exact values.

rounds	mean stage-game gap	max stage-game gap	max value error
100	0.00233	0.02020	0.01010
1000	0.00026	0.00296	0.00133
10000	0.00004	0.00094	0.00037

This is matrix-level fictitious play. The professor's point is that policy gradient itself does the solving, which is what section 5 tests.

8. Continuous actions: the dog game (provisional)

Policy gradient is applied to continuous actions directly: a network outputs an angle (von Mises) and a radius (scaled Beta) and is trained by self-play PPO. The dog-and-sheep game itself is **my placeholder**, since no definition exists yet, so these numbers say nothing about equilibrium.

matchup	capture rate	mean capture step
random dog vs random sheep	0.00	96
trained dog vs random sheep	0.26	41
random dog vs trained sheep	0.00	–
trained dog vs trained sheep	1.00	30
straight-line dog vs trained sheep	1.00	19

9. Assumptions and open questions

All 27 assumptions are listed, each marked as quoted, answered by the professor, mine or invented, in `docs/solver_assumptions.md`. Still open for the professor: the dog-game rules, the polar-policy distribution, what “check symmetry” should mean, which states to evaluate on, how degenerate equilibria should be compared, and two page-3 items (the “R game” closed form and “# time f is activated”).

Reproduce: `python scripts/pg_finite.py --seeds 3`, then `make pg-pdf`.